LAB 7 - FUZZING FOR INPUT VALIDATION BUGS

Movie Tickets Application | Hypothesis Property-Based Testing

Student	Debargho Banerji	**USN**	PES1UG24CS141
Repository	CODESTOCKFISH/L6_L7_15_movie_tickets	**Branch**	main
Application	Lab 7/movie_tickets.py	**Test file**	test_movie_tickets.py

1. Objective

To explore fuzz testing as a security and quality-assurance technique by using the Hypothesis library to identify input-validation bugs in the supplied Python application, correcting the defects, and checking the updated behaviour.

2. Functions Tested

Function	Purpose	Validation added
parse_seat()	Parses a seat label such as H12.	Checks input type, trims surrounding whitespace, and validates the row letter and numeric seat portion.
total_price()	Calculates the ticket total including GST.	Checks the input type and non-empty CSV string; validates integer conversion.
average_rating()	Calculates the mean of whitespace-separated ratings.	Checks the input type and non-empty string; validates integer conversion before averaging.

3. Testing Method

The test module uses Hypothesis with **st.text() | st.none()** to generate arbitrary text values and None. The three property-based tests pass these generated values to the relevant application functions. A controlled ValueError represents rejection of invalid input; unexpected exceptions fail the test. A separate deterministic test checks representative valid inputs.

The project-specific file names are **movie_tickets.py** and **test_movie_tickets.py**, corresponding to the generic application and test file names used in the lab handout.

4. Environment and Initial Application

The repository was opened locally and the Lab 7 directory was checked before the dependencies were installed.

```
PS C:\Users\Debargho Banerji\Downloads> cd L6_L7_15_movie_tickets
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets> cd "Lab 7"
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets\Lab 7> dir


    Directory: C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets\Lab 7


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        11-11-2026     12:42            959 movie_tickets.py
-a----        11-11-2026     12:42            303 README.md
-a----        11-11-2026     12:42            321 test_movie_tickets.py
```

Figure 1. Lab 7 directory showing movie_tickets.py, README.md and test_movie_tickets.py.

Hypothesis and pytest were installed with pip and verified using pip show.

```
[notice] A new release of pip is available: 26.2.1 -> 26.3
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets\Lab 7> python -m pip show hypothesis
Name: hypothesis
Version: 6.170.15
Summary: The property-based testing library for Python
Home-page: https://hypothesis.works
Author:
Author-email: "David R. MacIver and Zac Hatfield-Dodds" <david@drmaciver.com>
License-Expression: MPL-2.0
Location: C:\Users\Debargho Banerji\AppData\Local\Programs\Python\Python313\Lib\site-packages
Requires: sortedcontainers
Required-by:
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets\Lab 7> python -m pip show pytest
Name: pytest
Version: 9.2.0
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laugher, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: C:\Users\Debargho Banerji\AppData\Local\Programs\Python\Python313\Lib\site-packages
Requires: colorama, iniconfig, packaging, pluggy, pygments
Required-by:
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets\Lab 7>
```

Figure 2. Installed Hypothesis and pytest package information.

The original application was first executed using representative inputs H12, 150,250,400 and 4 5 3. The normal outputs were recorded before the code was changed.

```
==== Running parse_seat ====\Downloads\L6_L7_15_movie_tickets\Lab 7> python movie_tickets.py
Enter a seat number (e.g. H12): H12
Row and Seat: ('H', 12)

==== Running total_price ====
Enter ticket prices as CSV (e.g. 150,250,400): 150,250,400
Total with GST: 944.0

==== Running average_rating ====
Enter ratings separated by spaces (e.g. 4 5 3): 4 5 3
Average Rating: 4.0
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets\Lab 7>
```

Figure 3. Original Movie Tickets application execution with valid inputs.

5. Initial Fuzz Testing and Bugs Found

The initial Hypothesis run exposed edge cases that the original implementation did not handle safely. Hypothesis reduced the failing values to small examples, making the defects easier to identify.



Figure 4. Initial pytest run showing failing property-based tests and Hypothesis examples.

Function	Input	Failure observed
parse_seat()	Empty string ''	IndexError from accessing seat[0].
total_price()	None	AttributeError when calling split() on None.
average_rating()	Empty string ''	ZeroDivisionError because the parsed list is empty.
average_rating()	None	AttributeError when calling split() on None.

6. Corrected Application

Input validation was added to all three functions. Non-string inputs, empty input strings and malformed numeric tokens now raise a controlled ValueError. The GST multiplier remains 1.18 and the valid-input behaviour is preserved.

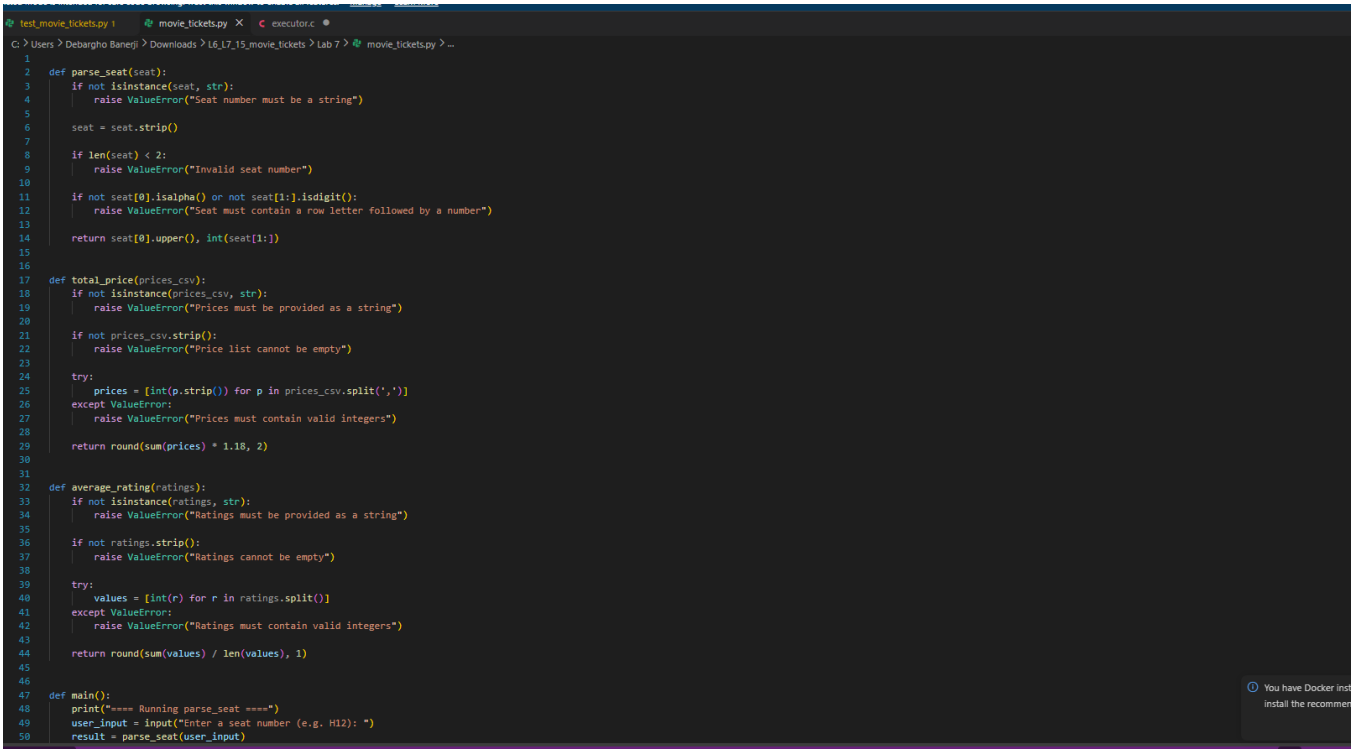


Figure 5. Updated movie_tickets.py showing validation in parse_seat(), total_price() and average_rating().

The main() function continues to prompt for the same three values and display the results. The corrected code keeps the original application flow while validating function inputs.

7. Updated Hypothesis Test Module

The completed test module contains three Hypothesis tests and one deterministic valid-input test. Each generated test accepts the deliberate ValueError used for invalid input and checks the result when the input is accepted. Unexpected exception types are not caught.

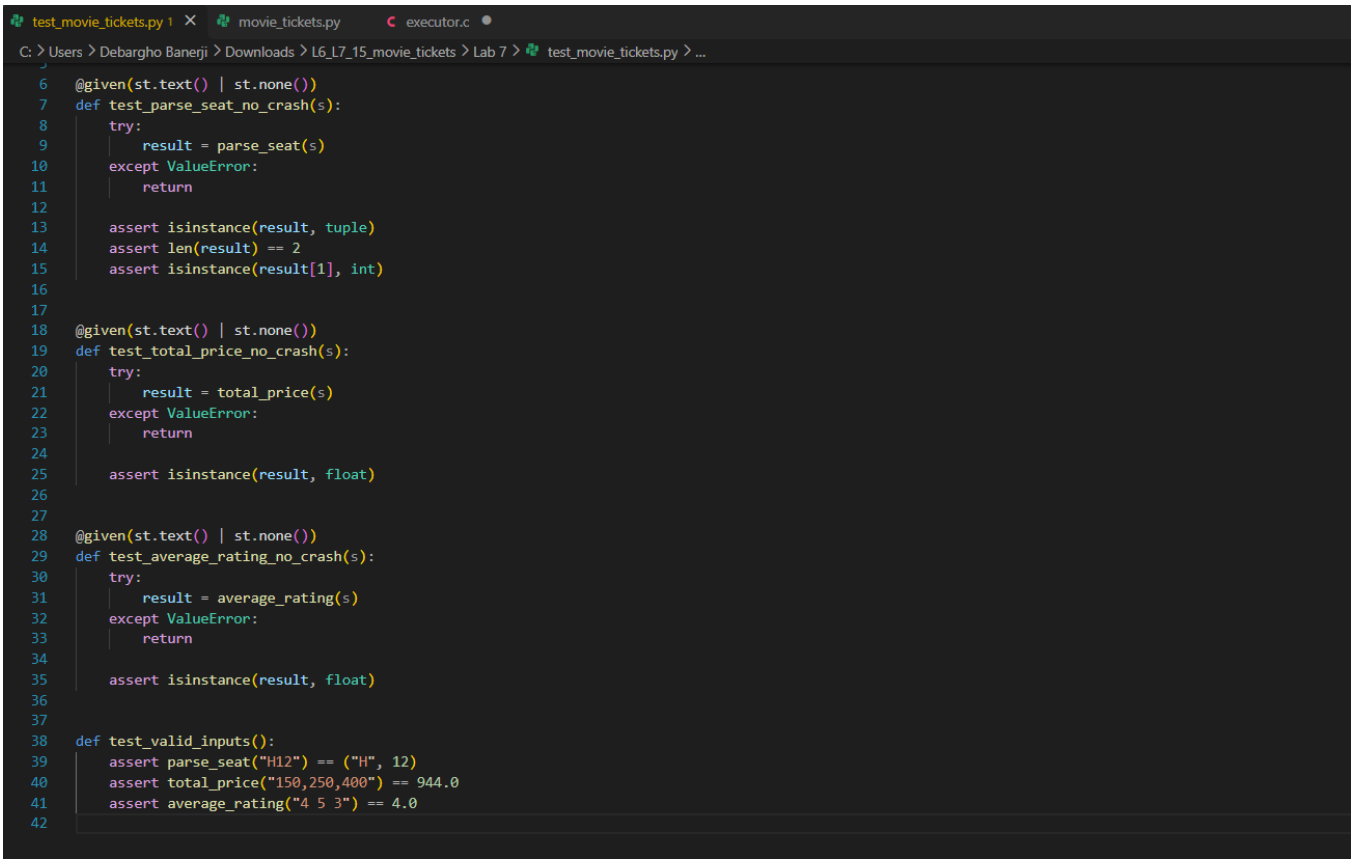


Figure 6. Updated test_movie_tickets.py with generated-input tests and valid-input assertions.

The deterministic examples verify the expected outputs:

parse_seat("H12") returns ("H", 12)
total_price("150,250,400") returns 944.0
average_rating("4 5 3") returns 4.0

8. Final Test Results and Application Execution

After the application and test module were updated, pytest was run using **python -m pytest -v test_movie_tickets.py**. Four tests were collected and all four passed in 1.17 seconds.

```
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets\Lab 7> python -m pytest -v test_movie_tickets.py
============================================================ test session starts ============================================================
platform win32 -- Python 3.13.5, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Debargho Banerji\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets\Lab 7
plugins: anyio-4.12.1, hypothesis-6.170.15
collected 4 items

test_movie_tickets.py::test_parse_seat_no_crash PASSED                                                                              [ 25%]
test_movie_tickets.py::test_total_price_no_crash PASSED                                                                             [ 50%]
test_movie_tickets.py::test_average_rating_no_crash PASSED                                                                          [ 75%]
test_movie_tickets.py::test_valid_inputs PASSED                                                                                     [100%]

============================================================ 4 passed in 1.17s =============================================================
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets\Lab 7>
```

Figure 7. Final pytest output showing four passing tests.

Test	Result
test_parse_seat_no_crash	**PASSED**
test_total_price_no_crash	**PASSED**
test_average_rating_no_crash	**PASSED**
test_valid_inputs	**PASSED**

The application was executed again with the same valid sample values to check that the interactive workflow and displayed results remained correct.

```
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets\Lab 7> python movie_tickets.py
==== Running parse_seat ====
Enter a seat number (e.g. H12): H12
Row and Seat: ('H', 12)

==== Running total_price ====
Enter ticket prices as CSV (e.g. 150,250,400): 150,250,400
Total with GST: 944.0

==== Running average_rating ====
Enter ratings separated by spaces (e.g. 4 5 3): 4 5 3
Average Rating: 4.0
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets\Lab 7>
```

Figure 8. Final execution of the corrected Movie Tickets application.

9. Version Control and Repository Update

The two modified Lab 7 Python files were committed on the main branch. The first push attempt to the SETAPESU26 repository returned HTTP 403 because the authenticated account did not have write access. The origin remote was changed to the personal repository CODESTOCKFISH/L6_L7_15_movie_tickets, after which the push completed successfully.

```
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets> git commit -m "Complete Lab 7 fuzz testing and input validation"
[main 4782e05] Complete Lab 7 fuzz testing and input validation
 2 files changed, 68 insertions(+), 2 deletions(-)
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets> git push origin main
remote: Permission to SETAPESU26/L6_L7_15_movie_tickets.git denied to CODESTOCKFISH.
fatal: unable to access 'https://github.com/SETAPESU26/L6_L7_15_movie_tickets.git/': The requested URL returned error: 403
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets> git remote -v
origin  https://github.com/SETAPESU26/L6_L7_15_movie_tickets.git (fetch)
origin  https://github.com/SETAPESU26/L6_L7_15_movie_tickets.git (push)
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets> git remote set-url origin https://github.com/CODESTOCKFISH/L6_L7_15_movie_tickets
.git
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets> git remote -v
origin  https://github.com/CODESTOCKFISH/L6_L7_15_movie_tickets.git (fetch)
origin  https://github.com/CODESTOCKFISH/L6_L7_15_movie_tickets.git (push)
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets> git push -u origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 1.32 KiB | 1.32 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/CODESTOCKFISH/L6_L7_15_movie_tickets.git
   f9a3c54..4782e05  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Debargho Banerji\Downloads\L6_L7_15_movie_tickets>
```

Figure 9. Git terminal output showing the corrected origin remote and successful push of commit 4782e05.

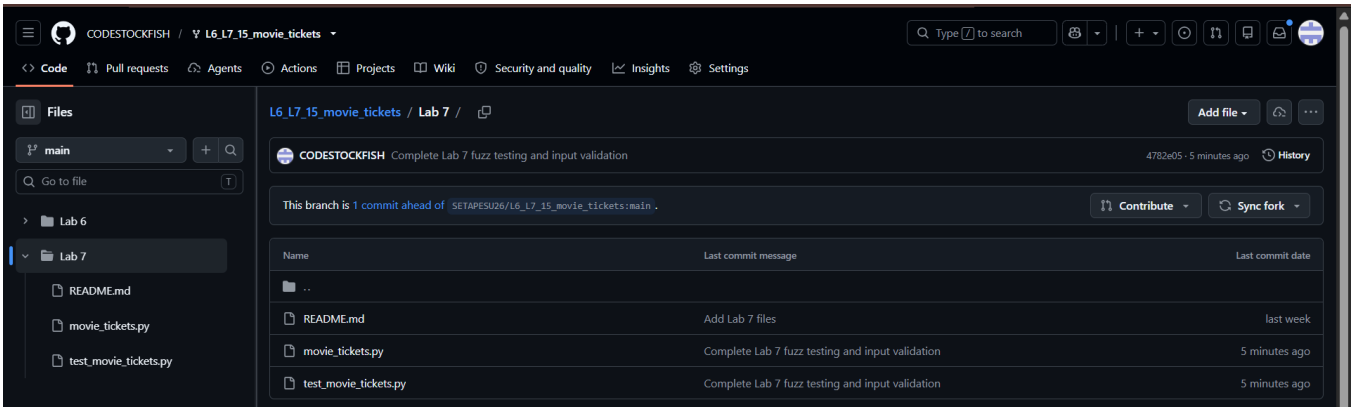


Figure 10. GitHub Lab 7 directory showing the updated application and test files.

10. Conclusion

Hypothesis fuzz testing exposed input-validation defects involving empty strings and None values. The three application functions were updated with defensive validation, and the final test run completed with four passing tests. The application continued to produce the expected results for the supplied valid examples. The updated Python files were committed and pushed to the personal GitHub repository.

Repository: https://github.com/CODESTOCKFISH/L6_L7_15_movie_tickets